

CI/CD for Machine Learning

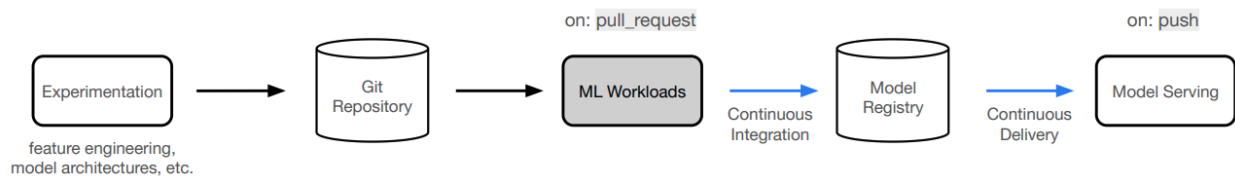
Using workflows to establish continuous integration and delivery pipelines to reliably iterate on our application.

Intuition

We want to be able to automatically execute our ML workloads with Jobs and Services when certain events occur (new data, performance regressions, elapsed time, etc.) to ensure that our models are always up to date and increasing in quality. In this lesson, we'll learn how to create continuous integration and delivery (CI/CD) pipelines to achieve an application that is capable of continual learning.

CI/CD

We're going to use [Jenkins](#) to create our CI/CD pipelines. It allows us to define workflows that are triggered by events (pull request, push, etc.) and execute a series of actions.



Events

Workflows are triggered by an **event**, which can be something that occurs on an event (like a [push](#) or [pull request](#))...

In our application, our `workloads` workflow is triggered on a pull request to the `main` branch and then our `serve` workflow and `documentation` workflows are triggered on a push to the `main` branch.

This creates for the following ideal workflow:

1. We make changes to our code and submit a pull request to the `main` branch.
2. Our `workloads` workflow is triggered and executes our model development workloads.
3. If the performance of the new model is better, we can merge the pull request and push the changes to the `main` branch.
4. Our `serve` workflow is triggered and deploys our application to production (along with an update to our `documentation`).

Continual learning

And with that, we're able to automatically update our ML application whenever we make changes to the code and want to trigger a new deployment. We have full control because we can decide not to trigger an event (ex. push to `main` branch) if we're not satisfied with the results of our model development workloads. We can easily extend this to include other events (ex. new data, performance regressions, etc.) to trigger our workflows, as well as integrate with more functionality around orchestration (ex. [Prefect](#), [Kubeflow](#), etc.), [monitoring](#), etc.

